

Обработка ошибок

Wrapping

Чтобы добавлять контекст к ошибке по мере её всплытия вверх по стеку вызовов, сохраняя при этом исходную ошибку для дальнейшего анализа.

```
package main

import (
    "fmt"
    "os"
)

func readFile(filename string) error {
    data, err := os.ReadFile(filename)
    if err != nil {
        // %w — это магический маркер для обёртывания
        return fmt.Errorf("не удалось прочитать файл %s: %w", filename, err)
    }
    fmt.Println(string(data))
    return nil
}

func main() {
    err := readFile("config.txt")
    if err != nil {
        fmt.Println("Ошибка:", err)
        // Вывод: Ошибка: не удалось прочитать файл config.txt: open config.txt: no
        such file or directory
    }
}
```

Ключевые моменты:

- Используйте только %w, не %v (иначе потеряете возможность распаковки)
- Создаётся цепочка ошибок, которую можно разворачивать

Проверка ошибок: errors.Is + errors.As

errors.Is – для сравнения с конкретным значением (глобальной переменной) Используется, когда ошибка – это известная глобальная переменная-сигнал.

```
package main

import (
    "errors"
    "fmt"
)

var ErrNotFound = errors.New("record not found")

func findUser(id int) error {
    if id == 0 {
        return fmt.Errorf("поиск пользователя: %w", ErrNotFound)
    }
    return nil
}
```

```
func main() {
    err := findUser(0)
    if errors.Is(err, ErrNotFound) {
        fmt.Println("Пользователь не найден (ожидаемая ситуация)")
    } else if err != nil {
        fmt.Println("Другая ошибка:", err)
    }
}
```

Когда использовать? Когда вы проверяете, что ошибка относится к конкретному типу сигнала (например, `io.EOF`, `sql.ErrNoRows`).

`errors.As` – для извлечения конкретного типа ошибки. Используется, когда вам нужно получить доступ к полям структуры ошибки.

```
package main

import (
    "errors"
    "fmt"
    "os"
)

func main() {
    _, err := os.Open("non-existent.txt")

    var pathErr *os.PathError
    if errors.As(err, &pathErr) {
        fmt.Printf("Ошибка операции: %s, путь: %s\n", pathErr.Op, pathErr.Path)
        // Вывод: Ошибка операции: open, путь: non-existent.txt
    }
}
```

panic vs error

error:

- Ожидаемые ситуации: файл не найден, нет прав, некорректный ввод
- Вы можете обработать ошибку на уровне выше

panic:

- Неисправимых состояний, где продолжение работы бессмысленно (программные баги)
- Инициализация: не удалось загрузить критический конфиг
- `nil`-указатель в месте, где этого не должно быть по логике

// ПЛОХО (не используйте `panic` для ожидаемых ошибок)

```
func divide(a, b int) int {
    if b == 0 {
        panic("деление на ноль") // Пользователь мог бы просто проверить b
    }
    return a / b
}
```

// ХОРОШО

```
func divide(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("деление на ноль невозможно")
    }
}
```

```
    return a / b, nil
}
```

Хороший пример:

```
func MustReadConfig(path string) *Config {
    data, err := os.ReadFile(path)
    if err != nil {
        panic(fmt.Sprintf("критическая ошибка: не удалось загрузить конфиг %s: %v",
path, err))
    }
    // ... парсинг
    return &Config{}
}

func main() {
    // Если конфиг не загружен — приложение не имеет смысла
    config := MustReadConfig("/etc/app/config.json")
    // ... запуск сервера
}
```

recover – перехват паники

Важно:

recover работает только внутри отложенной (defer) функции.

```
func safeCall(fn func()) {
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("Перехвачена паника:", r)
            // Можно залогировать, но не продолжать выполнение
        }
    }()
    fn() // если здесь panic — не убьёт программу
}

func main() {
    safeCall(func() {
        panic("что-то пошло не так")
    })
    fmt.Println("Программа продолжает работу")
}
```

Где применять:

- Веб-серверы: чтобы один упавший запрос не положил весь сервер
- Горутины: чтобы паника в одной горутине не обрушила всю программу

Пример:

```
// Пример: обработчик HTTP с защитой от паники
http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
    defer func() {
        if err := recover(); err != nil {
            log.Printf("panic: %v", err)
            http.Error(w, "internal server error", http.StatusInternalServerError)
        }
    }()
})
```

```
    // ... код, который может запаниковать
})
```

Цепочка обработки

```
package main

import (
    "errors"
    "fmt"
)

var ErrInvalidID = errors.New("invalid user ID")

type UserError struct {
    UserID int
    Msg    string
}

func (e *UserError) Error() string {
    return fmt.Sprintf("user %d: %s", e.UserID, e.Msg)
}

func findUser(id int) error {
    if id <= 0 {
        return fmt.Errorf("validation: %w", ErrInvalidID)
    }
    if id == 42 {
        return &UserError{UserID: id, Msg: "user is banned"}
    }
    return nil
}

func handleRequest(id int) error {
    err := findUser(id)
    if err != nil {
        return fmt.Errorf("обработка запроса: %w", err)
    }
    return nil
}

func main() {
    err := handleRequest(42)
    if err != nil {
        // Проверяем конкретный тип
        var ue *UserError
        if errors.As(err, &ue) {
            fmt.Printf("Ошибка пользователя: ID=%d, причина=%s\n", ue.UserID, ue.Msg)
        }

        // Проверяем сигнал
        if errors.Is(err, ErrInvalidID) {
            fmt.Println("Некорректный ID (валидация)")
        }

        // Выводим полную цепочку
        fmt.Println("Полная ошибка:", err)
    }
}
```

```
    // Вывод: Полная ошибка: обработка запроса: user 42: user is banned
  }
}
```